

- 1. Perform and document several tests of part 4 with at least 3 different common elements in both lists.**
 - a. I ran my `act_05.rb` script using three different values that appear in both `list1.txt` and `list2.txt`:
 - i. 20700
 - ii. 115604
 - iii. 432767
 - b. For each input, I observed the sort and search times using:
 - i. Recursive and tail-recursive versions of Quick Sort
 - ii. Recursive and tail-recursive versions of Merge Sort
 - iii. Recursive and tail-recursive versions of Binary Search
 - c. Using the Ruby VM to optimize for tail-recursion, I saw slightly better results across all test cases with the exception of Merge-Sort.
- 2. What is the fastest combination of sort and search algorithm? explain why do you think this happens?**
 - a. The fastest combination in my testing was QuickSort - Tail Recursive + Binary Search - Tail Recursive, with the exception of Merge Sort, which was noticeably slower in its tail-recursive form.
 - b. Quick Sort performed better than Merge Sort because it uses in-place sorting, meaning it sorts within the same array instead of creating new arrays. Binary Search (both recursive and tail-recursive) was consistently fast due to its $O(\log n)$ time complexity, and the tail-recursive version maintained that speed while also avoiding deep stack calls.
- 3. What is the longest palindrome in `palindromes_to_check.txt`.**

a. The longest palindrome is (only after removing special characters and spaces) is:

i. Doc, note, I dissent. A fast never prevents a fatness. I diet on cod.

4. How many palindromes of 3,4 and 5 words do exist in `palindromes_to_check.txt`.

a. 3-Word: 12

b. 4-Word: 26

c. 5-Word: 8